

Multihilo (Multithreading)

Lo definimos como el uso de múltiples hilos de ejecución que comparten un mismo proceso.

Los hilos pueden ejecutarse de forma paralela (en caso de que el sistema tenga múltiples núcleos de CPU) o bien de forma concurrente (si el sistema tiene un único núcleo, el sistema operativo va alternando entre hilos de ejecución).

El **multihilo** se usa para realizar tareas simultáneas como la lectura de archivos, la descarga de datos en segundo plano o la gestión de múltiples tareas de forma eficiente.

Ejemplo: Imagina que tenés una aplicación que necesita realizar varias tareas al mismo tiempo, como descargar archivos, procesar imágenes y realizar cálculos. Sin hilos, tendrías que hacer cada tarea una por una, lo que podría hacer que la aplicación se sienta lenta o no responda.

Usando hilos, cada tarea puede ejecutarse en paralelo o de forma concurrente, lo que mejora el rendimiento general y la capacidad de respuesta de la aplicación.

Ventajas de usar hilos



1 Eficiencia

Los hilos permiten realizar múltiples operaciones al mismo tiempo, lo que mejora la eficiencia en programas que requieren tareas paralelizadas.

2 Mejor uso de la CPU

En sistemas con múltiples núcleos de CPU, los hilos pueden ejecutarse en paralelo, aprovechando al máximo los recursos del hardware.

3 Interactividad

En aplicaciones como interfaces gráficas, los hilos permiten que la interfaz de usuario siga respondiendo mientras se realizan otras operaciones en segundo plano.

Desventajas de usar hilos



1 Sincronización

Como los hilos comparten el mismo espacio de memoria, pueden ocurrir problemas si dos hilos intentan acceder o modificar los mismos datos al mismo tiempo. Esto puede generar condiciones de carrera.

2 Complejidad

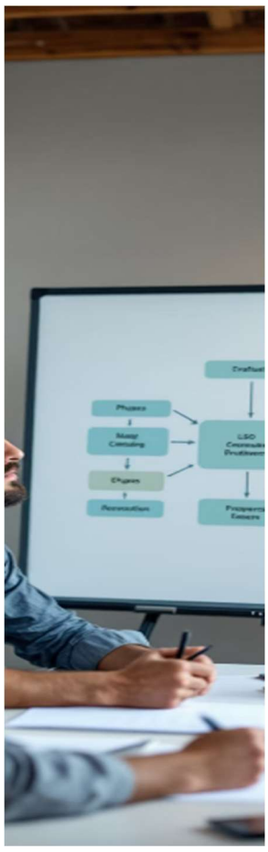
La programación con hilos puede ser más compleja que la programación secuencial, especialmente cuando se trata de coordinar varios hilos o gestionar el acceso a recursos compartidos.

3 Sobrecarga

Crear y gestionar muchos hilos puede generar una sobrecarga en el sistema, especialmente si los hilos no están bien distribuidos o si el programa crea más hilos de los que puede manejar eficientemente.

4 Memoria independiente

?



?

Un proceso es la definición para el sistema operativo de un programa en ejecución.

Se trata de una serie de instrucciones que se ejecutan secuencialmente haciendo uso de un conjunto de recursos del sistema, como memoria, ficheros y dispositivos de entrada y salida, etc.

Cada proceso tiene su propio espacio de memoria y se ejecuta de forma aislada de otros procesos, lo que significa que un proceso no puede acceder directamente a la memoria o a los recursos de otro proceso sin la intervención del sistema operativo.

Cada proceso tiene su propio espacio de memoria, lo que significa que no puede acceder directamente a la memoria de otro proceso. Esto ayuda a proteger a los procesos de interferencias y errores no deseados (por ejemplo, un proceso no puede sobrescribir la memoria de otro proceso sin causar un problema).



La memoria de un proceso se divide en diferentes segmentos como:

▼ **Código**

Instrucciones que ejecuta el proceso.

▼ **Datos**

Variables globales y estáticas.

▼ **Ligereza**

Los hilos son más ligeros que los procesos porque comparten el mismo espacio de memoria, lo que hace que crear y gestionar hilos sea más eficiente en términos de recursos que crear nuevos procesos.

▼ **Pila**

Usada para almacenar datos temporales y la información de las llamadas a funciones.

▼ **Heap (montículo)**

Área de memoria para la asignación dinámica de memoria en tiempo de ejecución (por ejemplo, mediante malloc en C o new en C++).

Contador de programa

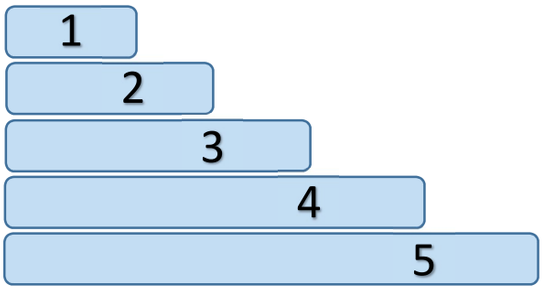
Un contador de programa es un registro que mantiene la dirección de la siguiente instrucción a ejecutar en el proceso. Cuando el proceso es ejecutado, este contador se va actualizando a medida que se van ejecutando las instrucciones.

Hilos de ejecución

Un proceso puede contener uno o más hilos (threads). En un sistema de multihilos, el proceso puede tener múltiples hilos que se ejecutan concurrentemente dentro del mismo espacio de memoria.

Estado del Proceso

Un proceso puede estar en diferentes estados según su ciclo de vida



1 Nuevo:

El proceso está siendo creado.

2 Ejecutando:

El proceso está siendo ejecutado por la CPU.

3 Esperando (o bloqueado):

El proceso está esperando que se cumpla alguna condición (por ejemplo, esperando entrada/salida).

4 Listo:

El proceso está esperando para ser ejecutado, pero la CPU está ocupado con otros procesos.

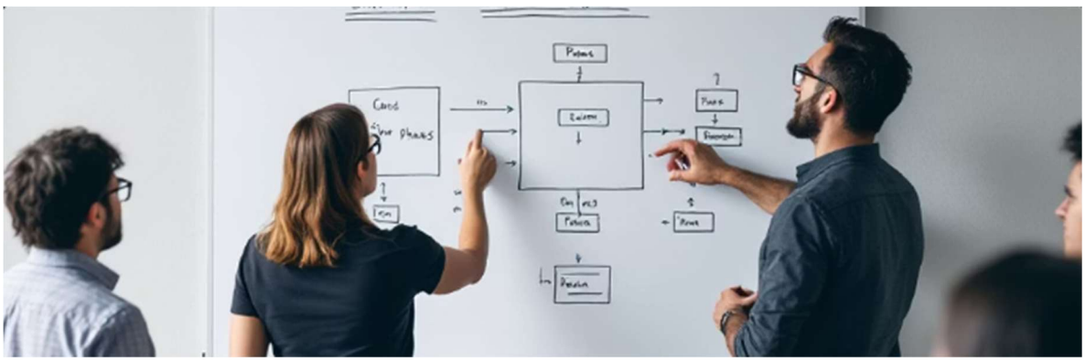
5 Terminado:

El proceso ha terminado su ejecución.



Identificador del proceso (PID)

Cada proceso tiene un identificador único asignado por el sistema operativo, conocido como PID (Process ID). Este identificador permite al sistema operativo diferenciar entre los distintos procesos en ejecución.



Diferencia entre proceso e hilo

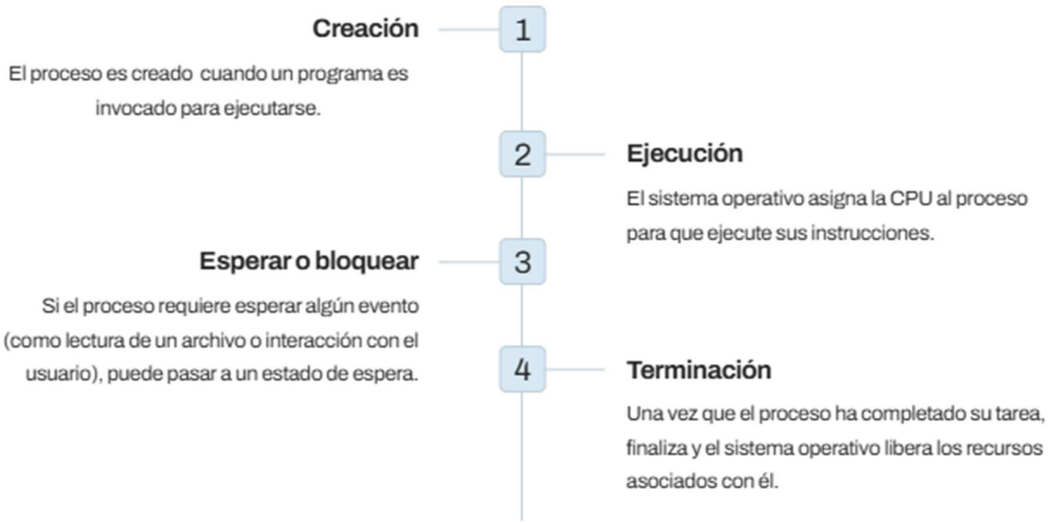
Proceso

Es una instancia independiente de un programa, con su propio espacio de memoria. Los procesos no comparten su memoria, a menos que se utilicen mecanismos específicos de comunicación entre procesos (IPC).

Hilo

Es una unidad de ejecución dentro de un proceso. Los hilos dentro de un mismo proceso comparten el mismo espacio de memoria y recursos del proceso, pero cada hilo tiene su propio contador de programa y pila de ejecución.

Ciclo de vida de un proceso



Ejemplo del ciclo de vida de un proceso

1. **Creación:** Un usuario hace doble clic en un ícono para ejecutar un programa, lo que crea un proceso.
2. **Ejecución:** El sistema operativo asigna un núcleo de la CPU al proceso para que empiece a ejecutar sus instrucciones (como leer datos de un archivo o calcular algo).
3. **Esperar:** El proceso puede estar esperando la respuesta de la red o de un dispositivo de entrada/salida, y se pone en estado de espera.
4. **Terminación:** Una vez completada la tarea, el proceso termina y el sistema operativo recoge la memoria y otros recursos.



¿Cómo se comunican los procesos?

Dado que cada proceso tiene su propia memoria, para que dos procesos se comuniquen entre sí, deben utilizar algún mecanismo de comunicación entre procesos (IPC). Algunas de las formas más comunes son:

Pipes (tuberías)

Permiten que un proceso envíe datos a otro proceso.

Memoria compartida

Dos o más procesos pueden compartir un bloque de memoria común para intercambiar datos.

Colas de mensajes

Permiten a los procesos enviar y recibir mensajes de forma asíncrona.

Sockets

Usados principalmente para la comunicación entre procesos que pueden estar en diferentes máquinas dentro de una red.

Ventajas de los procesos

1 **Aislamiento**

Como los procesos tienen su propia memoria, no pueden interferir directamente entre sí, lo que ayuda a evitar fallos catastróficos debido a errores de otro proceso.

2 **Seguridad**

Los procesos están aislados entre sí, lo que ayuda a mejorar la seguridad, ya que un proceso no puede acceder directamente a la memoria de otro proceso.

Desventajas de los procesos

1 **Consumo de recursos**

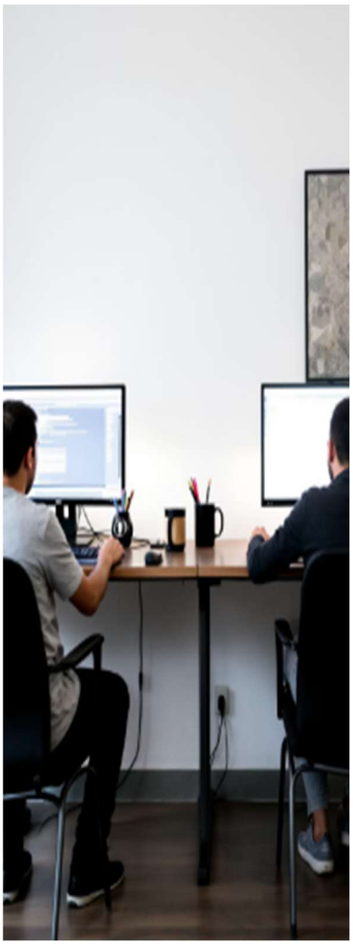
Crear y manejar procesos tiene un costo en términos de recursos del sistema. Los procesos son más pesados que los hilos, ya que cada uno necesita su propio espacio de memoria y recursos.

2 **Comunicación entre procesos**

La comunicación entre procesos puede ser más lenta y compleja que entre hilos, debido a la necesidad de utilizar mecanismos de IPC.

En resumen

Un proceso es una entidad independiente en ejecución dentro de un sistema operativo. Cada proceso tiene su propio espacio de memoria y recursos. Los procesos son gestionados por el sistema operativo y pueden contener múltiples hilos, que son unidades más pequeñas de ejecución dentro de esos procesos.



Ejercicio para la clase 2

Contador Concurrente

Imagina que estás desarrollando un programa en Python que necesita realizar un conteo de números del 1 al 10 de manera concurrente. Para ello, vas a utilizar dos hilos. El primer hilo se encargará de contar los números del 1 al 5, y el segundo hilo de contar los números del 6 al 10. Los números deben ser impresos en consola, y deben hacerlo de manera simultánea, es decir, ambos hilos deben trabajar en paralelo.

Tu tarea es escribir el código en Python para que estos hilos se ejecuten simultáneamente, de forma que ambos hilos impriman los números de forma intercalada.

Requerimientos:

1. Crear dos hilos que cuenten de forma simultánea.
2. Cada hilo debe contar hasta 5 números (uno por uno).
3. Los números deben imprimirse de forma concurrente (no necesariamente en orden secuencial).
4. Asegurarse de que los hilos se ejecuten de manera concurrente sin bloqueos entre ellos.